

Lab Report No 10
Understanding of Hill Climbing Search and A Star Search
Artificial Intelligence



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

27th May, 2025

Department of Computer Engineering,
HITEC University, Taxila

Example Task No 1:

Solution:

Brief description (3-5 lines)

A Star Search:

The A* search algorithm is a popular and widely used pathfinding algorithm in computer science. It's particularly useful for finding the shortest path between nodes in a graph, such as in navigation systems or game AI.
--

The code

<pre>import heapq # Graph structure: Node -> List of (Neighbor, Cost) graph = { 'S': [('B', 4), ('C', 3)], 'B': [('F', 5), ('E', 12)], 'C': [('D', 7), ('E', 10)], 'D': [('E', 2)], 'E': [('G', 5)], 'F': [('G', 16)], 'G': [] } # Heuristic values (h(n)) heuristic = { 'S': 14, 'B': 12, 'C': 11, 'D': 6, 'E': 4, 'F': 11, 'G': 0 } def a_star_search(start, goal): open_list = [] # Push starting node: (f = g + h, g, current_node, path_so_far) heapq.heappush(open_list, (heuristic[start], 0, start, [start])) while open_list: f, g, current, path = heapq.heappop(open_list) if current == goal: print("Path found:", path)</pre>

```

        print("Total cost:", g)
        return path
    for neighbor, cost in graph[current]:
        g_new = g + cost
        f_new = g_new + heuristic[neighbor]
        heapq.heappush(open_list, (f_new, g_new, neighbor, path + [neighbor]))
    print("No path found.")
    return None
# Run the search from 'S' to 'G'
a_star_search('S', 'G')

```

The results (Screenshot)

```

Path found: ['S', 'C', 'D', 'E', 'G']
Total cost: 17

```

Lab Task No 1:

Solution:

Brief description (3-5 lines)

Implement the A* search algorithm to find the shortest path between two points on a grid.

Task Description:

1.Setup:

Create a grid environment where each cell can either be traversable (empty) or blocked (obstacle).

2.Inputs:

- Grid size (rows and columns).
- Starting point coordinates (row and column).
- Goal point coordinates (row and column).
- Obstacle locations (list of coordinates).

3.Algorithm Implementation:

- Implement the A* search algorithm to find the shortest path from the starting point to the goal point while avoiding obstacles.
- Use Manhattan distance (or any other appropriate heuristic) as the heuristic function for estimating the distance from each cell to the goal.

4.Output:

- The shortest path from the starting point to the goal point.
- Visualization of the grid with the path highlighted.

The code

```

import heapq
import matplotlib.pyplot as plt
import numpy as np
class Node:
    def __init__(self, position, g=0, h=0, parent=None):
        self.position = position # (row, col)
        self.g = g # Cost from start to node
        self.h = h # Heuristic cost to goal
        self.f = g + h # Total cost
        self.parent = parent
    def __lt__(self, other):
        return self.f < other.f
def heuristic(a, b):
    # Manhattan distance
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    open_set = []
    heapq.heappush(open_set, Node(start, g=0, h=heuristic(start, goal)))
    closed_set = set()
    node_map = {}
    while open_set:
        current = heapq.heappop(open_set)
        if current.position == goal:
            path = []
            while current:
                path.append(current.position)
                current = current.parent
            return path[::-1] # Reverse path
        closed_set.add(current.position)
        neighbors = get_neighbors(current.position, rows, cols)
        for neighbor in neighbors:
            if grid[neighbor[0]][neighbor[1]] == 1 or neighbor in closed_set:
                continue
            g = current.g + 1
            h = heuristic(neighbor, goal)
            neighbor_node = Node(neighbor, g=g, h=h, parent=current)
            if neighbor in node_map and g >= node_map[neighbor].g:
                continue
            node_map[neighbor] = neighbor_node

```

```

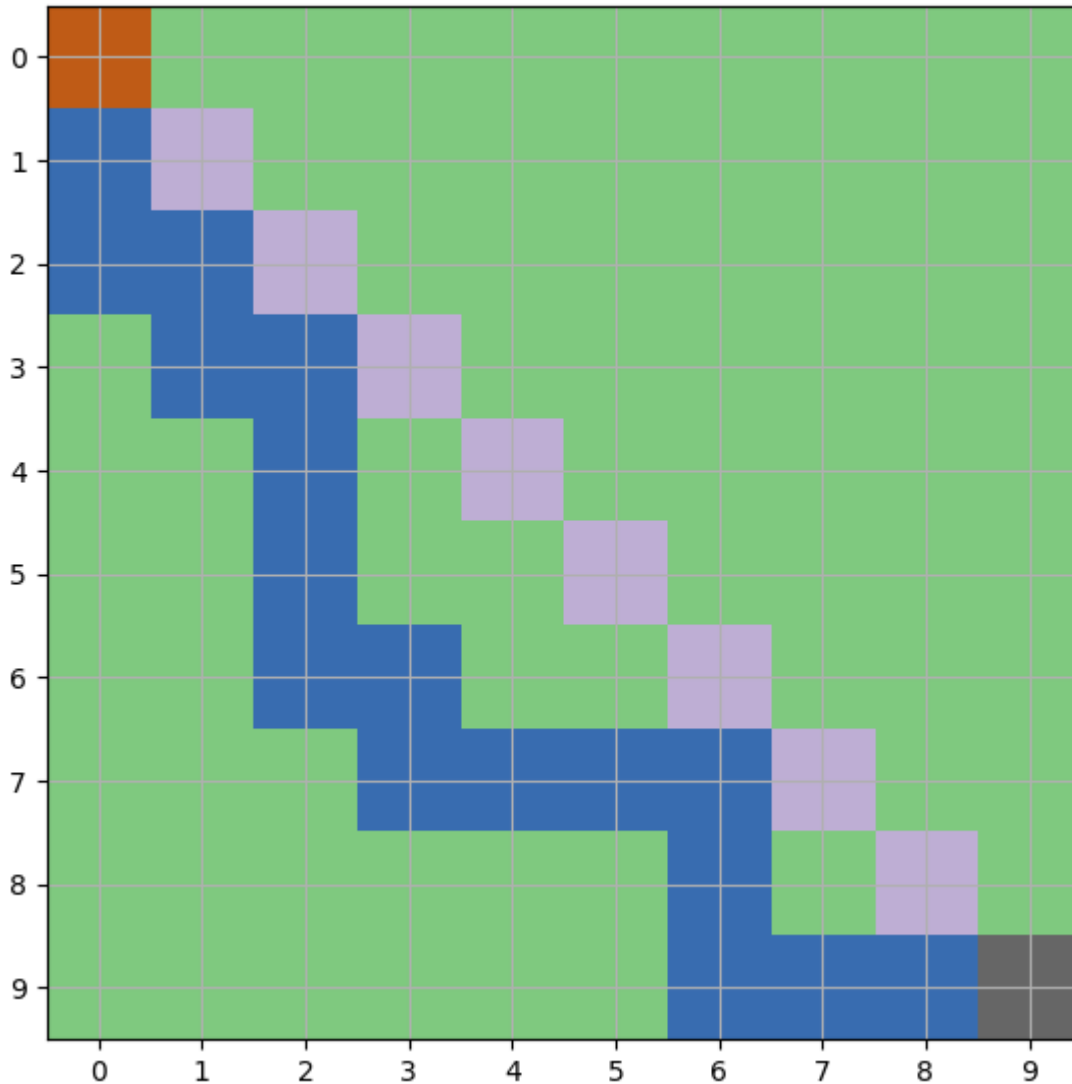
        heapq.heappush(open_set, neighbor_node)
    return None # No path found
def get_neighbors(pos, rows, cols):
    r, c = pos
    moves = [(0,1), (1,0), (0,-1), (-1,0)]
    neighbors = [(r+dr, c+dc) for dr, dc in moves]
    return [(nr, nc) for nr, nc in neighbors if 0 <= nr < rows and 0 <= nc < cols]
def visualize_grid(grid, path, start, goal):
    grid = np.array(grid)
    for r, c in path:
        if (r, c) != start and (r, c) != goal:
            grid[r][c] = 3 # Path color
    grid[start[0]][start[1]] = 4
    grid[goal[0]][goal[1]] = 5
    plt.figure(figsize=(6, 6))
    cmap = plt.cm.get_cmap('Accent', 6)
    plt.imshow(grid, cmap=cmap, origin='upper')
    plt.xticks(range(grid.shape[1]))
    plt.yticks(range(grid.shape[0]))
    plt.grid(True)
    plt.title("A* Pathfinding Visualization")
    plt.show()
# Example Usage
def main():
    rows, cols = 10, 10
    start = (0, 0)
    goal = (9, 9)
    obstacles = [(1,1), (2,2), (3,3), (4,4), (5,5), (6,6), (7,7), (8,8)]
    grid = [[0 for _ in range(cols)] for _ in range(rows)]
    for r, c in obstacles:
        grid[r][c] = 1
    path = astar(grid, start, goal)
    if path:
        print("Shortest path:", path)
        visualize_grid(grid, path, start, goal)
    else:
        print("No path found.")
if __name__ == "__main__":
    main()

```

The results (Screenshot)

Shortest path: [(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (5, 2), (6, 2), (6, 3), (7, 3), (7, 4), (7, 5), (7, 6), (8, 6), (9, 6), (9, 7), (9, 8), (9, 9)]

A* Pathfinding Visualization



Conclusion

In conclusion, the Hill climbing search is a greedy algorithm that moves towards a goal, while a search algorithm uses actual cost and heuristic estimate to find the most efficient path. Having a grasp of these algorithms allows for a balance between efficiency, accuracy, and computational cost in ai pathfinding.